BİL377 - HAFTA 02

Algoritmalar ve Doğruluk

Problem sözleşmesi, sözde kod ve döngü değişmezi

GİRDİ
SÖZLEŞME
ÇIKTI

Kavram, örnek, kod ve uygulama

Bu dersin sonunda

- Algoritma ile program arasındaki farkı açıklamak
- Ön koşul ve son koşul yazmak
- Sözde kodu izleyerek ara durumları kaydetmek
- Basit bir döngü değişmeziyle doğruluğu savunmak

BAŞARI GÖSTERGESİ

Kavramı tanımlar

Küçük bir örneği izler

Kodu çalıştırır

Maliyeti gerekçelendirir

Çıktılar ders sonundaki kısa kontrol sorularıyla yeniden ele alınacaktır.

Doğruluk neden testten fazlasıdır?

NEDEN?

- Testler yalnızca seçilmiş girdilerdeki davranışı gösterir.
- Ön koşul, algoritmanın hangi girdileri kabul ettiğini belirler.
- Döngü değişmezi bütün yinelemeleri kapsayan bir gerekçe kurar.

Temel soru

Bu bilgiyi kullanmadan algoritmalar ve doğruluk ile ilgili hangi kararı veremeyiz?

Doğruluk sözleşmesi

- Ön koşul, algoritmanın kabul ettiği girdileri sınırlar.
- Son koşul, döndürülen sonucun sağlaması gereken özelliği belirtir.
- Sözleşme iki taraflıdır: çağıran ön koşulu, algoritma son koşulu garanti eder.
- Eksik yazılmış son koşul, yanlış kodu da “doğru” gösterebilir.

ANA FİKİR

Ön koşul

n = len(values) ≥ 1

Son koşul

low, high ∈ values

her v için: low ≤ v ≤ high

Dikkat: yalnızca “low ≤ her değer” yazmak yetmez; **low = −∞** döndüren kod da bunu sağlar.

Kavramı tamamlayan ilkeler

- Algoritma her geçerli girdide son koşulu sağlamalıdır.
- Testler örnekleri kontrol eder, kanıt ise tüm geçerli girdileri kapsar.
- **Kısmi doğruluk:** algoritma biterse sonuç doğrudur.
- **Tam doğruluk** = kısmi doğruluk + sonlanma.

TEST

[7, 2, 9, 4] → (2, 9) ✓

Yalnızca bu girdi için bilgi verir.

KANIT

Her n ≥ 1 ve her values için

Değişmez + tümevarım → son koşul

Kavram sözlüğü

Ön koşul	Girdinin sağlaması gereken özellik
Son koşul	Çıktının sağlaması gereken özellik
Değişmez	Her yinelemede doğru kalan önerme
Sonlanma	Algoritmanın sonlu adımda bitmesi

Döngü değişmezi: başlangıç ve koruma

DEĞİŞMEZ I(k) **k. turdan sonra: low = min(values[0..k]) ve high = max(values[0..k])**

01 Başlangıç (k = 0)

Döngüden önce: **low = high = values[0]**

İşlenen önek: **values[0..0] = [values[0]]**

Tek elemanlı dizinin en küçüğü de en büyüğü de o elemandır.

⇒ **I(0) doğrudur.**

02 Koruma (k – 1 → k)

Varsay: **I(k – 1) doğru (tümevarım hipotezi)**

Döngü gövdesi: **low ← min(low, values[k])**

Çünkü: **min(values[0..k]) = min(min(values[0..k–1]), values[k])**

high için aynı akıl yürütme geçerlidir.

⇒ **I(k) doğrudur.**

Döngü değişmezi: sonuç ve sonlanma

DEĞİŞMEZ I(k) **k. turdan sonra: low = min(values[0..k]) ve high = max(values[0..k])**

03 Sonuç (döngü bittiğinde)

Son tur: **k = n – 1**

I(n – 1): **low = min(values[0..n–1]), high = max(values[0..n–1])**

values[0..n–1] dizinin tamamıdır.

⇒ low = min(values), high = max(values): son koşul.

04 Sonlanma

Döngü: **values[1:] üzerinde tam n – 1 tur**

Azalan ölçü: **kalan eleman n – 1 – k ≥ 0, her turda 1 azalır**

Negatif olamayan bir sayı sonsuza kadar azalamaz.

⇒ Kısmi doğruluk + sonlanma = tam doğruluk.

Doğruluğu taşıyan cümle

DEĞİŞMEZ

İşlenen önekte tutulan en küçük ve en büyük değerler doğrudur.

I(k): low = min(values[0..k]) ∧ high = max(values[0..k])

Başlangıç

İlk adımdan önce neden doğru?

Önek yalnızca values[0]; **low = high = values[0].**

Koruma

Bir adım daha atıldığında neden doğru kalır?

Yeni min = eski min ile yeni elemanın küçüğü; **low ← min(low, values[k])** tam bunu yapar.

Sonuç

Algoritma bittiğinde hangi sonucu verir?

k = n – 1'de önek tüm dizi: **low = min(values), high = max(values).**

Tahtada: kanıtın tamamı

ALGORİTMA

```
MinMax(values):
  // ön koşul: n ≥ 1
  low = high = values[0]
  for k = 1 .. n-1:
    low  = min(low,  values[k])
    high = max(high, values[k])
  return (low, high)
  // son koşul: low = min(values)
  //            high = max(values)
```

DEĞİŞMEZ I(k)

```
low  = min(values[0..k])
high = max(values[0..k])
```

KANIT

1 Başlangıç

k = 0: önek [values[0]], low = high = values[0] ⇒ I(0) ✓

2 Koruma

I(k−1) varsay; min(values[0..k]) = min(low, values[k]) ⇒ I(k) ✓

3 Sonuç

Döngü k = n−1'de biter; önek = tüm dizi ⇒ son koşul ✓

4 Sonlanma

n−1 tur; ölçü n−1−k her turda azalır ⇒ biter ✓

Kısmi doğruluk + sonlanma = tam doğruluk

Tek geçişte minimum ve maksimum

GİRDİ

Girdi: [7, 2, 9, 4]

BAŞLAMADAN ÖNCE

1. Beklenen çıktıyı tahmin et
2. İlk kararı belirle
3. İzlenecek durumu seç
4. Sınır durumunu düşün

Henüz sonucu göstermiyoruz; önce akıl yürütmeyi kuruyoruz.

Adım adım iz

k	values[k]	low	high	İşlenen önek	I(k)
0	7	**7**	**7**	[7]	✓
1	2	**2**	7	[7, 2]	✓
2	9	2	**9**	[7, 2, 9]	✓
3	4	2	9	[7, 2, 9, 4]	✓

Kalın kırmızı: o adımda değişen değer.

Sonuç: (low, high) = (2, 9) **Karşılaştırma:** 2 × (n − 1) = 6

HER SATIRDA SOR

Hangi durum değişti?

Hangi karar verildi?

Kontrol cümlesi doğru mu?

Sıradaki adım zorunlu mu?

Sonuç ve gerekçe

ÇIKTI

Çıktı: (2, 9)
Karşılaştırma sayısı 2(n – 1): doğrusal büyür

NEDEN DOĞRU?

İşlenen önekte tutulan en küçük ve en büyük değerler doğrudur.

Doğruluk kanıtı ile maliyet analizi farklı sorulara cevap verir.

Değişmezi kod içinde kontrol etmek


```
low = high = values[0]
for i, value in enumerate(values[1:], 1):
    low = min(low, value)
    high = max(high, value)
    assert low == min(values[: i + 1])
    assert high == max(values[: i + 1])
```

kod/algoritma_sozlesmesi.py

OKUMA SIRASI

1. Girdi ve başlangıç durumu
2. Karar noktası
3. Durum güncellemesi
4. Dönüş değeri

Kod inceleme ve testler

İNCELEME NOKTALARI

- assert öğretim ve hata ayıklama için kullanılır
- Dilim alma her turda kopya üretir: assert'li sürüm $O(n^2)$
- Kanıt fikri test kodundan bağımsız kalır

ASGARİ TEST KÜMESİ

Normal durum

Sınır durumu

Tekrarlı veya eşit değer

Geçersiz girdi davranışı

Bir satırı değiştir: hangi test başarısız olmalı?

Zaman ve bellek maliyeti

Zaman	**Her eleman bir kez işlenir: 2(n – 1) karşılaştırma → O(n).**
Bellek	**Yalnızca low ve high tutulur: ek bellek O(1).**
Yorum	Doğruluk kanıtı ile maliyet analizi farklı sorulara cevap verir.

Yaygın hatalar

01 Boş dizide ilk elemana erişmek

Bu hata hangi girdide görünür ve nasıl yakalanır?

02 Değişmezi döngü sonundaki sonuçla karıştırmak

Bu hata hangi girdide görünür ve nasıl yakalanır?

03 Sonlanmayı doğruluk kanıtının dışında bırakmak

Bu hata hangi girdide görünür ve nasıl yakalanır?

HATA

TEST

DÜZELTME

Test ve kanıtın rolleri

Örnek test	Belirli bir girdide beklenen sonucu denetler
Sınır testi	Boş, tek elemanlı veya yinelenen girdiyi denetler
Değişmez kanıtı	Tüm geçerli girdiler için genel gerekçe kurar

Seçim gerekçesi, yalnızca hız değil problem koşulları ve kaynak sınırlarıyla birlikte yazılır.

Sınıf içi uygulama

GÖREVLER

- İkili arama için gerekli ön koşulu yazın.
- Doğrusal aramanın döngü değişmezini bir cümleyle ifade edin.
- Boş girdi davranışına ilişkin bir tasarım kararı verin.

ÇÖZÜMDE ARANAN

Açık varsayım

İzlenebilir adımlar

Sınır durumu

Maliyet gerekçesi

Cevap kadar gerekçe de paylaşılır.

Ders sonu kontrolü

BUGÜN

- Sözleşme geçerli girdi ve doğru çıktıyı tanımlar.
- Değişmez başlangıç, koruma ve sonuç adımlarında kullanılır.
- Sonlanma ayrıca gerekçelendirilir.
- Testler kanıtı destekler fakat onun yerini almaz.

KENDİNİ SINAMA

Kavramı tanımlayabiliyor muyum?

Örneği kâğıt üzerinde izleyebiliyor muyum?

Maliyeti gerekçelendirebiliyor muyum?

Tek cümle: Bu haftanın en önemli fikri nedir?

Sözleşmeli dizi özeti

GÖREV

- Tek geçişte min, max ve toplam hesaplayan bir fonksiyon yazın.
- Ön koşul, son koşul ve döngü değişmezini rapora ekleyin.
- Boş, tek elemanlı ve yinelenen değerli girdileri test edin.

TESLİM

- rapor.md
- dizi_ozeti.py
- test_dizi_ozeti.py

DEĞERLENDİRME

Doğruluk 40
Analiz 25
Testler 20
Açıklık 15

Ayrıntılar aynı hafta klasöründeki ODEV.md dosyasındadır.